# EMAIL SPAM CLASSIFIER

A Beginner-Friendly Machine Learning Project

| | |
|---|---|
| Project type | Software-only / Natural Language Processing |
| Programming language | Python |
| Main methods | TF-IDF + Naive Bayes |
| Suggested interface | Streamlit web app |

Project guide with problem statement, dataset, system design, implementation code, evaluation, testing, and future scope.

## 2. Abstract

Email Spam Classifier is a machine learning project that identifies whether a message is spam or not spam (ham). Spam messages may contain unwanted advertisements, scams, suspicious links, or misleading offers. The system learns patterns from examples of labelled messages and predicts the category of a new message.

The project uses Python, text processing, TF-IDF feature extraction, and a Multinomial Naive Bayes model. A smallStreamlit web page lets a user paste a message and receive a prediction. This is an educational demo, not a perfect security filter.

## 3. Introduction

Email is used for communication, education, shopping, and work. However, unwanted messages can waste time and may try to trick people into sharing information or clicking unsafe links. A spam classifier can help by automatically sorting messages into likely spam and non-spam categories.

The model does not understand every message like a person. It learns statistical patterns from training examples, so data quality and regular testing are important.

## 4. Problem Statement

Manually checking every email takes time, and unwanted messages may be missed. The aim is to develop a software system that analyses the text of a message and predicts whether it is spam or non-spam.

## 5. Objectives

- Collect a labelled dataset of spam and non-spam messages.
- Clean and convert text into numeric features suitable for machine learning.
- Train a classification model and evaluate it on unseen examples.
- Build a simple interface where users can paste a message.

- Explain errors and limitations, especially false positives and false negatives.

# 1. Project Scope

- Software-only project; no special hardware is required.
- The first version classifies message text supplied by the user.
- It does not automatically connect to or change a real email account.
- The classifier can later be integrated with an email system only after privacy and security review.

# 1. Existing System and Proposed System

## Existing approach

Users may manually read messages, rely on built-in email filters, or block senders. These approaches can help, but some unwanted messages still reach the inbox.

## Proposed approach

The proposed system learns from labelled examples. It converts message text into features, predicts spam or non-spam, and shows the result in a web interface. It is a demonstration classifier and should not be treated as a guarantee that a message is safe.

# 2. Requirements

## Hardware

- Laptop or desktop computer.
- At least 4 GB RAM recommended.
- Internet access to install packages and download a dataset.

## Software

- Python 3.10 or newer.
- VS Code, PyCharm, or Jupyter Notebook.
- Libraries: pandas, scikit-learn, joblib, streamlit.
- A labelled message dataset in CSV format.

# 1. Dataset

A beginner can use the UCI SMS Spam Collection dataset, which contains labelled SMS messages. It is not a dataset of full email messages, so the report must describe it accurately as an SMS-based prototype. For a true email spam project, use a suitable email corpus such as the Apache SpamAssassin public corpus, following its usage terms and data-handling guidance.

The sample code expects a CSV file with two columns: 'label' and 'text'. Labels should be mapped to 'spam' and 'ham' (non-spam). If the downloaded dataset has different column names or label values, adapt the loading section.

### Example dataset format

| label | text |
|---|---|
| ham | Hi, are we still meeting tomorrow? |
| spam | Congratulations! Claim your prize now... |

Do not include personal or confidential messages without permission. Remove sensitive information when preparing custom datasets.

## 2. Methodology

- Collect labelled messages.
- Inspect the data and remove empty or duplicate rows where appropriate.
- Split the dataset into training and test sets while preserving the class balance.
- Convert text into numeric features using TF-IDF.
- Train a Multinomial Naive Bayes classifier.
- Evaluate using precision, recall, F1-score, confusion matrix, and accuracy.
- Save the trained pipeline.
- Build a Streamlit interface for testing new messages.

## 1. System Architecture

```
User enters message text
|          v Text validation
|         v TF-IDF vectorizer
(text -> numeric  features)
|          v Trained spam
classifier         |         v
Spam / Non-spam prediction          |         v
Display result and safety reminder
```

The vectorizer and classifier should be saved together in one pipeline. This ensures new messages are transformed in the same way as training messages.

## 2. Step-by-Step Implementation

### Step 1: Create a project folder

```
email_spam_classifier/ ■■■ data/
■   ■■■ messages.csv
■■■ models/
■   ■■■ spam_pipeline.joblib
■■■ train_model.py
■■■ app.py
■■■ requirements.txt
■■■ README.md
```

### Step 2: Install packages

```
python -m  venv .venv # Activate the environment, then run: pip
install pandas scikit-learnjoblib streamlit
```

### Step 3: Prepare the CSV

Place your dataset at data/messages.csv. It must contain 'label' and 'text' columns. Use 'spam' for spam and 'ham' for non-spam. Update the code if your dataset uses different names or labels.

## Step 4: Train and evaluate the model

```
# train_model.py from pathlib import Path import joblib import pandas as pd from
sklearn.model_selection import train_test_split from sklearn.pipeline import
Pipeline from sklearn.feature_extraction.text import TfidfVectorizer from
sklearn.naive_bayes import MultinomialNB from sklearn.metrics import
(    accuracy_score, classification_report, confusion_matrix )

df = pd.read_csv ("data/messages.csv") df = df.dropna(subset=[ "label", "text"])
df["label"] = df["label"]. astype(str). str.lower(). str.strip() df["text"] =
df["text" ]. astype(str). str.strip() df = df[ df["label"]. isin(["spam", "ham"])]  df =
df[ df["text" ] ! = ""]

X = df["text"] y =
df["label"]

X_train, X_test, y_train, y_test = train_test_ split(
   X, y, test_size=0.2, random_state =42, stratify= y )

pipeline = Pipeline([    ("tfidf",
TfidfVectorizer(       lowercase=True,
stop_words ="english",       ngram_range =( 1, 2),
max_features=30000     )),
   ("classifier",MultinomialNB()) ] )

pipeline.fit(X_train, y_train) pred =
pipeline.predict(X_test)

print("Accuracy:", round(accuracy_ score(y_test, pred), 4))  print("Classification report:")
print(classification_report (y_test, pred, zero_division=0))  print("Confusion matrix:")
print(confusion_matrix(y_test, pred, labels =[ "ham", "spam"]))

Path("models"). mkdir(exist_ok=True)  joblib.dump(pipeline, "models/
spam_pipeline.joblib") print("Saved model to models/
spam_pipeline.joblib")
```

This is a starter template. Check that your dataset labels are correct and the data is suitable. Do not report performance numbers until you have run the code on your selected dataset.

## Step 5: Build the web interface

```
# app.py import joblib import
streamlit as st

pipeline = joblib.load("models/spam_pipeline.joblib")

st.title("Email Spam Classifier") st.write("Paste a message below to check whether it looks like
spam.")

message = st.text_area ("Message text", height= 180)

if st.button ("Classify message"):    if not
message.strip():
      st.warning("Please enter a message first.")    else:
      prediction = pipeline.predict([message ])[ 0]       if prediction ==
"spam":
        st.error("Prediction: Likely SPAM")       else:
        st.success("Prediction: Likely NON-SPAM ")       st.caption(          "This is an
automated estimate. Do not click links or "          "share personal information based only on
this result."
      )
```

## Step 6: Run the project

```
python train_model.py  streamlit run
app.py
```

Open the local address shown in the terminal, usually http://localhost:8501. Keep the terminal running while you use the app.

# 3. Evaluation Metrics

- Accuracy: percentage of all test messages classified correctly.
- Precision for spam: among messages predicted as spam, how many really are spam.
- Recall for spam: among actual spam messages, how many the model finds.
- F1-score: combines precision and recall into one measure.

- Confusion matrix: shows correct and incorrect predictions for each class.

For spam filtering, precision and recall are both important. A false positive means a genuine message is wrongly labelled spam; a false negative means spam is missed. Do not use accuracy alone, especially if one class is much more common.

## 1. Testing Plan

| Test | Input | Expected result |
|---|---|---|
| Spam-like text | A sample promotional or scam-like mes | sageModel may label it spam; verify against actual output |
| Normal message | A routine personal message | Model may label it non-spam; verify actual output |
| Empty input | Submit with no text | Warning asks user to enter a message |
| Unseen wording | A message unlike training examples | A prediction is returned, but it may be wrong |
| Very long message | A long pasted message | App should respond or provide a clear error |

## 2. Expected Outcome

The completed project should train a text classifier, show evaluation metrics on held-out data, save the pipeline, and display a likely spam/non-spam result for text entered by the user. Actual performance will depend on the dataset and how similar future messages are to training examples.

## 3. Advantages

- Automatically screens message text quickly.
- Uses free, widely used Python tools.
- Simple enough for a beginner team to understand and demonstrate.
- Can be extended with more data and improved models.

## 1. Limitations

- New scam styles may not match the training data.
- Text-only models may miss clues in sender identity, headers, attachments, and URLs.
- A genuine message can be wrongly marked as spam.
- An unsafe message can be wrongly marked non-spam.
- A dataset of SMS messages may not represent real email messages.

## 1. Future Enhancements

- Train and evaluate on a true email corpus.
- Compare Naive Bayes with Logistic Regression and linear Support Vector Machines.
- Add explainability, such as showing words that influenced a prediction.
- Test carefully for class imbalance and data leakage.

- Add safe URL analysis and email-header features using controlled datasets.
- Integrate with an email client only after security, privacy, and user-consent review.

## 1. Conclusion

Email Spam Classifier is a practical beginner machine learning project. It introduces text preprocessing, TF-IDF, classification, evaluation, and simple web application development. The system can help demonstrate automatic message filtering, but it cannot guarantee that a message is safe. A suitable dataset and honest testing are essential.

## 2. References and Learning Resources


- Scikit-learn documentation: https://scikit-learn.org/
- TfidfVectorizer documentation:
https://scikit-learn.org/stable/modules/generated/sklearn.feature_extraction.text.TfidfVectorizer.html
- MultinomialNB documentation:https://scikit-learn.org/stable/modules/generated/sklearn.naive_bayes.MultinomialNB.html
- Streamlit documentation: https://docs.streamlit.io/
- UCI Machine Learning Repository — SMS Spam Collection:https://archive.ics.uci.edu/dataset/228/sms+spam+collection
- Apache SpamAssassin public corpus: https://spamassassin.apache.org/old/publiccorpus/